



A N T H R O P I C

CLAUDE / OS

YOUR AI
OPERATING SYSTEM
BUILT TO SHIP.



BUILD.

| AUTOMATE.

| SCALE.



FOUNDER EDITION

HERE'S EVERYTHING.

You commented MAP. Here's the complete system.

Every file in the .claude/ folder explained with real code.

Part 1: Technical — every file, what it does, why it matters.

Part 2: Agency Blueprint — what this looks like for a real business.

— PART 1 — TECHNICAL

THE FULL STRUCTURE

Four files live at your project root. Everything else lives inside .claude/.

Project Structure

YOUR-PROJECT/

■■■ CLAUDE.md	← brain. loads every session.
■■■ CLAUDE.local.md	← your private machine notes.
■■■ .gitignore	← keeps secrets off GitHub.
■■■ .mcp.json	← connects Claude to external tools.
■■■ .claude/	← everything below loads on demand.
■■■ skills/	← reusable workflows. /name to trigger.
■■■ agents/	← sub-agents with own context window.
■■■ hooks/	← shell scripts that ALWAYS fire.
■■■ rules/	← scoped rules per file path.
■■■ output-styles/	← controls Claude response format.
■■■ settings.json	← control panel. hooks need this.
■■■ settings.local.json	← private overrides. gitignored.

Key rule:

Root files (CLAUDE.md, .mcp.json, .gitignore, CLAUDE.local.md) must be at the project root. ...

— FILE 01

CLAUDE .MD

The brain. Loads every session. Claude knows your project before you type a word. Hard limit: 200 lines.

CLAUDE.md

CLAUDE.md — under 200 lines always

Project

Name: YourApp | Goal: one sentence | Stack: Next.js, Supabase

Rules

- Write production-ready code from day one
- Never install a package without confirming
- All DB ops through Supabase client only
- Every API route needs rate limiting + auth middleware

Architecture → See: docs/architecture.md

API Rules → See: .claude/rules/api.md

What We Never Use

- No Prisma (Supabase client directly)
- No Lucide icons (use Phosphor)

FILE 02

MCP.JSON

Wires Claude directly into external tools. No more screenshots. Claude reads the source.

Without MCP:

Screenshot → paste → Claude guesses → verify → repeat. 15 min per question.

With MCP: 'Users today?' → Claude queries DB → answers in 3 seconds.

.mcp.json

```
{ "mcpServers": {  
  // Supabase — reads + writes DB in plain English  
  "supabase": { "command": "npx",  
    "args": ["-y", "@supabase/mcp-server-supabase@latest"],  
    "env": { "SUPABASE_ACCESS_TOKEN": "$SUPABASE_TOKEN" } },  
  // GitHub — PRs, issues, repos inside chat  
  "github": { "command": "npx",  
    "args": ["-y", "@github/github-mcp-server"],  
    "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "$GH_TOKEN" } },  
  // Playwright — controls a real browser  
  "playwright": { "command": "npx",  
    "args": ["@playwright/mcp@latest"] }  
}
```

Security:

All keys use \$ENV_VAR syntax — pulled from environment, never hardcoded. The file is safe to...

FILE 03

HOOKS /

Shell scripts that fire at specific moments — no matter what. CLAUDE.md can be overlooked. Hooks cannot.

CLAUDE.md IS A SUGGESTION. HOOKS ARE LAW.

Runs at shell level. Zero exceptions.

session-start.sh (Trigger: SessionStart)

```
#!/bin/bash
echo "Branch      : $(git branch --show-current)"
echo "Last commit: $(git log -1 --format='%h %s (%cr)')"
echo "Uncommitted: $(git status --short | wc -l) file(s)"
if [ "$(git status --short | wc -l)" -gt 0 ]; then
  echo "■■■ Uncommitted changes in working tree"
fi
```

pre-deploy-guard.sh (Trigger: PreToolUse — Bash(vercel*))

```
npx tsc --noEmit
if [ $? -ne 0 ]; then echo "■ TypeScript errors"; exit 1; fi
npx next lint
if [ $? -ne 0 ]; then echo "■ Lint errors"; exit 1; fi
npm run test -- --passWithNoTests
if [ $? -ne 0 ]; then echo "■ Tests failing"; exit 1; fi
echo "■ All checks passed. Deploying..."
```

FILE 04

SKILLS /

Reusable multi-step workflows triggered by one slash command. Write once. Use everywhere. Global install.

Instead of a 10-step prompt every time —

save it as a skill. Call it with /skill-name. Claude runs the entire workflow.

```
skills/ship.md → /ship
```

```
# Full production deploy pipeline in one command  
Step 1: npx tsc --noEmit      # type check  
Step 2: npx next lint        # lint  
Step 3: npm run test          # unit tests  
Step 4: npm run build         # production build  
Step 5: vercel --prod         # deploy  
On success: show the deployment URL.  
On failure: stop, show exact error, do not continue.
```

```
skills/commit.md → /commit
```

```
1. git diff --staged  
2. Analyse the changes  
3. Write conventional commit: type(scope): description  
   Example: "fix(auth): resolve token refresh on 401"  
4. git commit -m '[message]' → show commit hash
```

FILE 05

AGENTS + RULES

Sub-agents run in isolated context windows. Deep work in isolation. Only a summary returns to your main session.

```
agents/debugger.md
```

```
---  
name: debugger  
description: Isolate bugs without polluting main session  
tools: Read, Grep, Glob, Bash  
---  
  
You are a debugging specialist. Find root cause, not symptoms.  
Output: Root Cause | Failure Path | Fix | Test Case
```

```
rules/api.md – fires only in src/app/api/**
```

```
---  
paths: - "src/app/api/**"  
---  
  
Every route: rate limiting + auth + Zod validation  
Error format: { error, code, status }  
Never: log request bodies | return raw DB errors
```

settings.json – the control panel

```
{ "model": "claude-opus-4-5",  
  "hooks": {  
    "SessionStart": [{ "command": "bash .claude/hooks/session-start.sh" }],  
    "PreToolUse":    [{ "matcher": "Bash(vercel*)", "command": "...guard.sh" }],  
    "PostToolUse":   [{ "matcher": "Write", "command": "...format.sh" }]  
  },  
  "permissions": { "allow": ["mcp__supabase__*", "Bash(npm run *)"],  
    "deny": ["Bash(git push --force)", "Bash(rm -rf *)"] } }
```

NOW LET'S SEE IT IN A REAL BUSINESS.

The structure above is universal.

Here's what it looks like deployed for an agency with real clients, real deadlines, and real money on the line.

REAL BUSINESS BLUEPRINT

WHITFIELD CREATIVE

James Whitfield runs a 12-person content agency in Austin, Texas. Good clients, solid revenue. Team spending most of their day on work a system should be handling.

The message that kicked this off:

"We spend 40hrs/week on first drafts, reformatting for clients, briefing writers on brand voice. I tried ChatGPT — someone still has to babysit every output. It's not saving anything."

— James Whitfield, Founder, Whitfield Creative, Austin TX

whitfield-creative/ – full structure

whitfield-creative/

```
■■■ CLAUDE.md           ← agency rules, client list, delivery workflow
■■■ .mcp.json           ← Notion + Airtable + Google Docs + Firecrawl
■■■ .claude/
  ■■■ skills/
    ■■■ blog-post.md    ← /blog-post [client] [topic]
    ■■■ social-pack.md  ← /social-pack – 5 formats from one brief
    ■■■ repurpose.md      ← /repurpose – one piece into everything
  ■■■ hooks/
    ■■■ session-start.sh ← loads today's Airtable content queue
    ■■■ pre-delivery.sh  ← QC before anything goes to client
  ■■■ rules/
    ■■■ techflow-inc.md  ← B2B SaaS brand voice
    ■■■ nova-wellness.md ← DTC wellness, UK English
    ■■■ ridge-financial.md ← B2B fintech brand voice
```

rules/techflow-inc.md – loads when 'TechFlow' is mentioned

```
--- paths: - "**/*techflow*" ---
Who They Are: B2B PM SaaS. Ops directors at 50-500 person cos.
Voice: Confident, plain language, outcome-focused.
'Save 6 hours/week' beats 'increase operational efficiency'
Their words: 'visibility', 'async', 'ship faster', 'reduce blockers'
Never: 'synergy' / 'leverage' / 'circle back' / competitor shots
CTA: 'Start your free trial' or 'See TechFlow in action'
```

hooks/pre-delivery.sh – blocks bad content reaching clients

```
#!/bin/bash # Fires before content marked ready for client
BANNED=('game-changer' 'cutting-edge' 'dive into' 'delve into')
for phrase in ${BANNED[@]}; do
  grep -qi $phrase $FILE && echo '■ Banned phrase' && FAILED=1
done
[ $WORDS -lt 800 ] && echo '■ Blog too short' && FAILED=1
! grep -qi 'book|schedule|try' $FILE && echo '■ No CTA' && FAILED=1
[ $FAILED -eq 1 ] && exit 1 || echo '■ QC passed.'
```

WHAT THE SYSTEM REMOVES:

Re-briefing writers on brand voice every project → Automated
Brand knowledge walking out when staff leave → Stays in system
Bad drafts reaching clients (manual QC) → Hook blocks it
Onboarding a new client taking weeks → One rules file
Needing more headcount to scale output → Add a skill instead

Honest framing:

Every hour your team spends re-briefing AI, fixing brand voice errors, or manually QC-ing dr...

— START HERE

SETUP CHECKLIST

New project. Do these in order. 20–30 minutes. Saves weeks.

✓ Create CLAUDE.md — /init in root. Name, goal, stack, rules. Under 200 lines.

■ Create CLAUDE.local.md — Local port, DB creds, shortcuts. Add to .gitignore.

■ Configure .mcp.json — DB, GitHub, Playwright at minimum. Keys in .env only.

■ Write session-start.sh — Branch + last commit every session. Register in settings.json.

■ Write pre-deploy-guard.sh — Type → lint → test → build. exit 1 on failure.

■ Configure settings.json — Register all hooks. Set allow/deny. Required for hooks to fire.

■ Create output-styles/terse.md — No preamble. Direct answers. Saves tokens.

■ Build your first skill — Your most repeated workflow. /skill-name forever.

The actual gap:

Everyone using Claude Code has the same model. The builders getting 10X output aren't better...

BUILDING THIS IN PUBLIC.

I share every system I build — architecture, code, failures. No gatekeeping.

Follow: [@akashafter2am on Instagram](#)

Building AI systems from Delhi → for businesses worldwide.